

The Journal as Substrate: Unifying Deployment, Replication, Backup, and Offline Operation in Distributed Systems

Alvaro Rivera

2026-05-14

Abstract

This paper is a design theory contribution in the sense of Alan Hevner, Stuart March, Jinsoo Park, and Sudha Ram (2004). It identifies a fragmentation that the canonical distributed-systems literature has documented across four separate operational disciplines without recognising them as instances of a single structural condition. It then derives the property under which this fragmentation dissolves and presents an instantiation demonstrating that the alternative is realisable in production.

For decades, deployment without downtime, cross-datacenter replication, backup, and offline operation have evolved as four parallel bodies of practice, each with its own vocabulary, apparatus, and cost model. This paper shows that this separation is contingent rather than structural. Under the substrate condition established in prior papers of this series — where the journal is homoiconic, dense, and composed of compact named operations rather than serialised states — the four disciplines reduce to a single primitive.

Deployment becomes replay against the substrate up to the present head. Replication is the same replay at another site. Backup is the corpus held without replay. Offline operation is replay after a delay.

The canonical apparatuses developed for these disciplines — blue-green environments, change-data-capture pipelines, snapshot agents, and message-queue buffers — perform work that the substrate already provides by construction. Each arose independently to compensate for the absence of the substrate condition rather than to extend it.

The construct introduced here names the substrate, formalises the four equivalences that follow from it, and characterises the operational regimes in which these equivalences can be observed.

Operations from the substrate

TL;DR

The canonical distributed-systems literature treats four operations — deployment without downtime, cross-datacenter replication, backup, and offline catch-up — as separate disciplines. Each has developed its own mature apparatus: blue-green and rolling updates for deployment; change-data-capture and log shipping for replication; snapshots and point-in-time recovery for backup; message queues and convergence protocols for offline operation. Systems that require all four must implement and operate all four independently.

This paper shows that the separation is contingent. Under the journal-as-substrate condition established in prior papers of this series — where the journal records operations rather than states, is written in the same language as the program, and consists of compact named entries — the four disciplines reduce to a single primitive.

Deployment is replay to the present head. Replication is the same replay at another site. Backup is the corpus held without replay. Offline operation is replay after a delay. The canonical apparatuses become structurally subsumed: each reconstructs, from outside the program, work the substrate already performs by construction.

Six laboratories measure the substrate under conditions where these equivalences hold. A version handover reaches the present head at 451k entries/s; cross-site transfer carries 36–99.7 bytes per event against payloads 3–9× larger; passive consumers converge across heterogeneous backends (FileSystem, MySQL, SQL Server) to identical in-memory state in 18/18 cells; local append latency runs 3× below co-located RDBMS engines under matched durability. §7 shows how four runtime primitives — each present for independent structural reasons — compose into the operations the canonical literature implements as separate subsystems.

For a journaled program, deployment is replay, replication is sharing history, backup is copying the program, and offline operation is delayed replay.

1. Introduction

This paper makes a design theory contribution. It identifies a structural property whose absence has been implicitly assumed by canonical distributed-systems literature, but never named as a single construct; derives the equivalences that follow when the property is present; and presents an instantiation showing it is realisable in production. The contribution is conceptual; the instantiation

serves as an existence proof rather than as the substance of the claim. The genre is that described by Alan Hevner, Stuart March, Jinsoo Park, and Sudha Ram (2004) as design science research: an artefact satisfying the construct’s conditions, augmented by measurements that exhibit the régime under which the construct holds.

This is neither a systems paper proposing a new mechanism nor a survey of existing apparatuses. It does not introduce a deployment tool, a replication protocol, a backup format, or an offline-operation pattern. It examines a structural property that prior literature has implicitly assumed absent and shows that, under its presence, four operational disciplines of distributed systems reduce to a single primitive. In this genre, contribution is measured by the precision of the construct, the validity of the equivalences derived from it, and the realisability of the instantiation. Sections 5 and 7 exhibit the instantiation and report measurements establishing the régime in which the equivalences are observable.

Distributed systems must address four operational dimensions: release without downtime, cross-datacenter replication, recovery after data loss, and periods of disconnectivity (offline operation). The canonical approach treats these as separate disciplines, each with distinct apparatus and cost semantics: blue-green and rolling updates; change-data-capture and log shipping; snapshots and point-in-time recovery; message-queue buffers and convergence protocols. These bodies of practice are mature and widely adopted, yet they evolved largely independently. Systems that require all four must implement and operate four distinct stacks, each with its own engineering and cost.

The fragmentation is rarely questioned because it is familiar. But the four disciplines can be restated more simply: they vary when and where a program is read — at the present head and in place (deployment), at the present head at another site (replication), held at another site without reading (backup), and read after a delay (offline catch-up). They differ along two axes — temporal position and spatial position — over a single artefact. If that artefact were the program itself, recorded as operations rather than states, the four disciplines would be four readings of the same primitive: reading the program in order.

This paper names that artefact and the equivalences it admits. The construct introduced is the journal as the substrate of the program. Under the anti-porosity condition of Paper 1 and the separability condition of Paper 2, the journal records operations rather than states, is written in the same language as the program, and reduces each entry to a compact reference to a parametric verb and its arguments. The journal, so constituted, is not a record kept by the program; it is the program written out over time.

Four equivalences follow. Deployment is replay to the present head. Replication is the same replay at another site. Backup is the corpus held without replay. Offline operation is replay after a delay. The canonical apparatuses above are responses to the absence of the substrate condition; under its presence they become structurally subsumed by the substrate.

§2 traces the genealogy of the four disciplines as parallel bodies of literature. §3 names the assumption that produced the fragmentation. §4 states the substrate theorem and the four equivalences. §5 develops each equivalence and reports measurements exhibiting the régime under which it holds. §6 examines four canonical apparatuses — blue-green deployment, change-data-capture, snapshot backup, and message-queue buffer — and shows that each reconstructs, from outside the program, work the substrate already performs. §7 presents an instantiation in production. §8 relates the construct to prior papers in this series. §9 addresses counter-arguments. §10 concludes.

2. Genealogy: four separate operational disciplines

For decades, four operational concerns — deployment downtime, cross-datacenter replication, backup, and offline operation — have evolved as separate bodies of literature and practice. Each developed its own vocabulary, its own apparatus, and its own cost model. The genealogies traced below are not exhaustive; they are sufficient to establish that the canon treats these four as distinct disciplines, each with a dedicated mechanism, and that the resulting fragmentation of the operational stack has not been challenged within any of the four traditions.

2.1 Deployment downtime

The earliest deployment practice in production systems was the cutover: stop the running service, swap binaries, start it again. Downtime was accepted as the cost of release. The first patterned response was articulated by Fowler in *BlueGreenDeployment* [Fowler 2010], which named a procedure in which “you ensure that you have two production environments, as identical as possible” and traffic is switched from one to the other at release time. The mechanism is environmental: two parallel deployments, one apparatus for routing, one for binary distribution.

The continuous-delivery literature ratified the pattern. Humble and Farley [Humble & Farley 2010] generalized blue-green into a broader release-engineering discipline in which deployment is a build-pipeline output rather than an operator action, and downtime is a regression to be detected rather than a default to be accepted.

The modern instantiations are canary release [Sato 2014] and the orchestration-mediated rolling update [Burns et al. 2016; Kubernetes documentation]. Canary release introduces partial traffic exposure before full cutover, treating the deployed binary as a hypothesis tested against a fraction of production load. The Kubernetes rolling update replaces pods incrementally under a readiness-probe gate, making the deployment apparatus a property of the orchestrator rather than of the application.

A parallel strand of the deployment canon addresses schema and data migration. Ambler and Sadalage [Ambler & Sadalage 2006] formalized *database refactoring* as a discipline in which every change to the data store is recorded as a forward-and-backward pair of scripts versioned alongside the application code. The pattern is instantiated by migration frameworks such as Rails Migrations, Liquibase [Liquibase project], and Flyway [Flyway project], which apply versioned scripts to the data store at release time. The migration apparatus is separate from the traffic-switching apparatus: it runs adjacent to — or before — the cutover, mutates the store in place, and produces a schema or data shape that the next binary version expects.

Across these mechanisms the shape is constant: a separate apparatus is built whose function is to absorb the change — of binary, of schema, or both — without interrupting the service. The apparatus is external to the program; the program is unaware that it is being replaced or migrated.

2.2 Cross-datacenter replication

Replication across datacenters originates in database engines. MySQL replication [MySQL Reference Manual] established the canonical primary/replica topology in the 1990s: the primary writes a binary log of statements or row-level events, and replicas tail that log. The replication mechanism lives inside the database engine; the application program is unaware that replication is occurring.

The pragmatic maturation came with change-data-capture (CDC). Debezium [Debezium project] and Maxwell [Maxwell project] read the database’s binary log and re-emit each change as an event on an external transport, typically Kafka. CDC inserts an apparatus between the database and downstream consumers: a process that watches state changes in one system and reconstructs them as event streams for others. The reconstruction step is necessary precisely because the originating system does not emit events — it emits state changes that CDC observes and translates.

The modern instantiations extend the apparatus topologically. Cassandra’s multi-datacenter replication [Lakshman & Malik 2010] embeds the replication topology in the storage engine itself, allowing writes accepted in any datacenter to propagate to peers under a configurable consistency level. Kafka MirrorMaker [Kreps, Narkhede, & Rao 2011] generalizes the pattern for log-structured systems: a dedicated process consumes from one cluster and re-produces to another, with offsets tracking progress per topic.

Across these mechanisms the shape is constant: cross-datacenter replication requires a separate apparatus — engine-internal, engine-adjacent, or fully external — whose function is to observe the changes happening in one location and reproduce them elsewhere.

2.3 Backup

The backup canon predates the others. Agent-based backup [Bacula project; Veeam documentation] developed in the 1990s and 2000s as a discipline in which a dedicated process periodically reads the production data store and writes a copy to an archival medium. The agent is external to the application; backup occurs at the storage layer.

The pragmatic maturation introduced snapshot semantics. The WAFL filesystem [Hitz, Lau, & Malcolm 1994] established the modern primitive: a point-in-time copy taken at the filesystem level, exploiting copy-on-write so that the snapshot is cheap and consistent with respect to the moment of capture. Database engines adopted the same pattern under the name *point-in-time recovery* [PostgreSQL documentation], in which continuous archival of the write-ahead log permits reconstruction of any committed state within the archived range.

The modern instantiations move the apparatus to object storage. Cloud-era backup [AWS S3 Lifecycle; Azure Blob Storage backup documentation] treats archival as a property of the storage tier rather than of a dedicated agent: lifecycle policies move data to colder tiers automatically, and recovery is a matter of retrieval from the archive.

Across these mechanisms the shape is constant: backup is a separate apparatus that produces a copy of state and is exercised at recovery time to reconstruct that state. The backup procedure observes the data store from the outside; the program being backed up does not participate.

2.4 Offline operation

The offline-operation canon is the most heterogeneous of the four, because it spans network-layer, transport-layer, and application-layer treatments of disconnection. The early precedent is store-and-forward messaging: UUCP and, later, SMTP queue undeliverable messages locally and retry until the remote endpoint is reachable. Disconnection is absorbed by a queue at the boundary of the system.

The pragmatic maturation generalized the boundary queue into a programmable apparatus. RabbitMQ [RabbitMQ documentation] introduced dead-letter exchanges to capture messages whose delivery is impeded, deferring them for retry or human inspection. Kafka [Kreps, Narkhede, & Rao 2011] generalized the pattern further: consumer offsets are explicit, durable, and per-partition, so a consumer that goes offline resumes from its last committed offset when it returns. The queue is no longer a remediation mechanism for failed delivery; it is the substrate over which the consumer’s progress is tracked.

The modern instantiations carry the pattern into application-level state. Eventually-consistent stores [DeCandia et al. 2007] treat each replica as a local source of truth that converges with peers when connectivity permits. The Dynamo paper established the design vocabulary — vector clocks, hinted

handoff, read-repair — that subsequent systems (Cassandra, Riak, Redis replication [Redis documentation]) instantiate in varying degrees.

Once again, the shape is constant: offline operation requires a separate apparatus — a queue, an offset, a convergence protocol — whose function is to absorb the period during which the system cannot reach its counterpart.

2.5 Where the operational complexity lives

The four disciplines surveyed above can be compared along a single matrix. Each row names a discipline; each column names a dimension of the apparatus the canon developed to handle that discipline.

Discipline	Where the complexity lives	Apparatus	Cost
Deployment	Outside the program, in the release pipeline	Two-environment switch (blue-green); incremental pod replacement under a readiness gate (rolling update); fractional traffic exposure (canary); versioned schema and data migration scripts (Liquibase, Flyway, Rails Migrations)	Duplicate production environment; orchestration layer; manual or automated cutover procedure; forward-and-backward migration pairs
Cross-datacenter replication	Inside the storage engine, or in an adjacent CDC layer	Binary-log shipping (engine-internal); change-data-capture (engine-adjacent); multi-DC topology (engine-aware)	Replication lag; reconstruction overhead in CDC; consistency-versus-availability configuration surface

Discipline	Where the complexity lives	Apparatus	Cost
Backup	Outside the program, in a dedicated archival pipeline	Agent-based copy; filesystem or engine snapshots; continuous log archival (PITR); object-store lifecycle	Recovery-time objective and recovery-point objective; storage-tier cost; periodic exercise of the recovery procedure
Offline operation	At the boundary between system and counterpart	Persistent queue with retry semantics; durable consumer offsets; convergence protocol over replicated stores	Buffer sizing; offset management; eventual-consistency reasoning at the application layer

Three observations follow from the table. The location of the complexity differs across rows: the deployment apparatus lives in the release pipeline, the replication apparatus lives inside or beside the engine, the backup apparatus lives in a separate archival pipeline, and the offline-operation apparatus lives at the system’s boundary. The mechanism in each row was developed without reference to the others: the blue-green literature does not cite the CDC literature; the snapshot literature does not cite the consumer-offset literature. The four literatures evolved in parallel, each solving what it believed to be a different problem. And the cost in each row is paid separately: a system that wishes to be zero-downtime, cross-datacenter, recoverable, and tolerant of disconnection budgets independently for each of the four columns.

The canon, taken as a whole, presents these four as four different problems requiring four different apparatuses. §3 names the assumption that made this fragmentation appear necessary.

3. The assumption named

What §2 shows to be continuous across four separate bodies of literature can be stated as a single structural assumption:

Operational concerns of a system are addressed by apparatus around the program rather than by the program itself.

Across the four disciplines, the response to each operational concern is the introduction of a mechanism external to the program. Blue-green deployment builds

a parallel environment to absorb the cut-over. Change-data-capture inserts a translation layer between stores to carry replication. Snapshot agents observe the data store from outside and write captures to archival media. Message-queue brokers hold messages at the system boundary to absorb periods of disconnection. In each case, the mechanism is not a sentence of the program it serves. It observes the program from outside, maintains a parallel artefact, and is operated independently of the program’s own execution.

The assumption embedded in these practices is that operational concerns require such external apparatuses, each developed independently to address a specific condition.

The alternative this assumption leaves unexplored is that the program might address these concerns by varying how it is read — by changing when or where the program is replayed, without leaving the program’s own representation. Under this alternative, the four disciplines become four readings of a single primitive. The question becomes structural: under what condition would the program admit such readings?

§4 names that condition and the four readings it permits. The apparatuses surveyed in §2 remain appropriate responses in their original context; what this paper shows is that they arise from the absence of the substrate condition, which, if present, subsumes the need for each apparatus.

4. The substrate theorem

4.1 The substrate

The first two papers in this series established two structural properties of the journal that this paper takes as preconditions. Paper 1, §3 named *anti-porosity*: the journal records the operations of the program rather than the states they produce, and admits no representational sparsity at the boundary between domain code and persisted form. The journal is therefore homoiconic — entries are sentences in the same language the program is written in — and dense — every effect that crosses the boundary is named in that language, not summarised as state delta. Paper 2, §1.2 and §3 named *separability*: programs that are parametric in their arguments admit a compiled form in which the operation’s body is cached under an identifier and the journal entry reduces to that identifier plus the call’s arguments. Together the two conditions yield a journal whose entries are compact, named operations rather than serialised states.

Under these two conditions the journal is not a record kept by the program; it is the program written out over time. Each entry is a sentence the program has uttered; the journal is the corpus of those sentences in the order they were uttered. The naming move of this paper is to call this artifact *the substrate of the program*: the journal does not store the program’s outputs, it constitutes the program’s existence over time. The program and the journal are the same

artifact under two readings — one synchronous, one diachronic. Under the synchronous reading, the runtime is the program. Under the diachronic reading, the journal is the program.

4.2 The theorem statement

The theorem of this paper can be stated as a single proposition.

Given a program whose execution is recorded as a journal satisfying the anti-porosity condition of Paper 1 and the separability condition of Paper 2 — that is, a journal that is homoiconic, dense, and made of compact named operations rather than serialised states — the following four equivalences hold.

E1 — Deployment is replay. A new version of the runtime, started from the substrate, reconstructs the program by reading the journal sequentially and applying each entry. Nothing about the new version’s instantiation differs in kind from what the previous version did at every entry up to that point; the act of bringing a process to a state in which it can serve requests is the act of replaying the substrate up to its present head. Deployment therefore differs from steady-state operation only in the position of the read cursor over the same corpus.

E2 — Replication is sharing history. A second instance of the program, running on a different machine or in a different site, reconstructs the program by reading the same sequence of entries the first instance wrote. The act of replicating is the act of transmitting the substrate from one site to another and replaying it at the destination; no state extraction, no schema mapping, and no change-data inference enter the act. Replication therefore differs from deployment only in the site at which the same corpus is replayed.

E3 — Backup is copying the program. A consumer that reads entries from the substrate and persists them locally — without instantiating the runtime that would replay them — holds a copy of the program in the form in which it was written. The act of backing up is the act of duplicating the substrate; recovery is the act of replaying the duplicate. Backup therefore differs from replication only in whether the destination chooses to replay the same corpus or merely to hold it.

E4 — Offline operation is delayed replay. A consumer that is unreachable when entries are written, and that catches up by reading them later from a persistent buffer or directly from the source, applies the same replay to the same sequence as a consumer that received the entries in real time. The act of operating offline is the act of replaying the substrate after a delay; the delay does not enter the program, only the cursor of the consumer. Offline operation therefore differs from replication only in the elapsed time between the write and the read of the same corpus.

The four equivalences are not parallel restatements of a single mechanism. Each

addresses one classical operation of distributed systems (§2) and shows that the operation, as the canon presents it, dissolves into a single primitive — replay — once the substrate’s properties are taken as given. E1 addresses the time axis (a new version replacing an old one in place); E2 addresses the space axis (a new site running the same program); E3 addresses the storage axis (a consumer holding the corpus without running it); E4 addresses the connectivity axis (a consumer reading later than the writer wrote). The four together exhaust the operational variations that the canonical literature in §2 treats as separate disciplines. The operational specifics of how each replay is realised — handoff, transmission, passive ingestion, catch-up — are taken up in §5; §4 confines itself to the structural claim.

4.3 Apparatus: the substrate and its four replay arrows

The shape of the theorem can be rendered as a simple spatial arrangement. No new content is introduced; the arrangement makes the four equivalences of §4.2 visible at a glance, with the substrate at the centre and the four conditions arrayed around it.

The substrate — the journal as program — sits at the centre. From it radiate four equivalences, each corresponding to one classical operational discipline:

- **E1 — Deployment:** replay to the present head.
- **E2 — Replication:** replay of the same corpus at another site.
- **E3 — Backup:** the corpus held without replay.
- **E4 — Offline operation:** replay after a delay.

The vertical pair (E1, E4) varies the temporal position of the replay — at the present head, or after a delay. The horizontal pair (E2, E3) varies the spatial position of the corpus — replayed at a different site, or merely held at one. The canonical literature in §2 names four disciplines by looking at the destinations; viewed from the substrate, only replay occurs, under four conditions on when and where.

5. Consequences: four equivalences as variants of replay

Each of the four sub-sections that follow develops one equivalence of §4.2. The sub-section opens with the operational realisation of the equivalence — the structural intervals through which the program is read under that variant — and follows with a measurement that exhibits the régime in which the equivalence holds. Each lab is designed to isolate the structural content of its equivalence, not to optimise throughput; performance figures are reported as evidence of the régime, not as benchmark claims.

5.1 E1 — Deployment is replay

§4.2 stated the equivalence: a new version of the runtime, started from the substrate, reconstructs the program by reading the journal sequentially and applying each entry. §5.1 develops the operational realization of this equivalence — the handover window during which an instance about to become authoritative reads the substrate up to the present head, and the precise sense in which that window is replay and nothing else.

The handover window A new version of the runtime begins as a *follower*: a process that reads from the same substrate the live instance writes but does not yet accept requests. The follower’s task is to reach the present head of the substrate. Once it has — and once a brief synchronization tail has cleared — the follower is admitted as authoritative and the previous version is released. Three structural intervals comprise the window:

1. **Bulk replay.** The follower opens the journal from entry zero (or from a checkpoint, if the substrate is partitioned with one) and applies each entry to its in-memory state in the order written. The state reconstructed at the end of bulk replay equals, entry-for-entry, the state the previous version reached by writing those same entries.
2. **Lock and synchronization tail.** The previous version is paused briefly so that no further entries are written. The follower reads the small residue of entries that arrived between the start of bulk replay and the pause — the *handover tail* — and applies them. The state at the end of the tail equals the state the previous version held at the moment it was paused.
3. **Gate flip.** The follower is admitted as the authoritative instance. From this point on the substrate’s writer is the new version; the previous version is released.

The operational specifics of each interval are realized by a single primitive in the instantiation (§7.1 develops the gate mechanism); what §5.1 establishes is that the *content* of the window is replay. No state extraction, no schema migration, and no inference step enters the operation. The new version becomes authoritative by reading what the previous version wrote, in the order it was written, until the read cursor reaches the position the previous version had left it at.

What the substrate measures L1 (lab paper05-lab1-redblack-replay-time @ ef3a002) instruments the handover window over four journal sizes (N {1k, 10k, 100k, 1M} entries) under the compact-action régime named in Paper 2. The measurement isolates the inner bulk-replay interval and the handover tail separately so that the structural content of the window — the act of replay — is reported in its own right rather than aggregated with orchestration overhead.

Under the compact-action régime with `AlwaysCompiled` compilation, the substrate replays N entries at a sustained rate of **451 thousand entries per**

second (p50, $N = 100\,000$), completing the bulk replay in **221.6 ms** (p95 = 237.4 ms). The rate is reached as the runtime amortises between $N = 10\,000$ and $N = 100\,000$ entries and holds through the $N = 1\,000\,000$ anchor at 416 thousand entries per second. The handover tail is below 30 μs at every N measured. **Bulk replay accounts for more than 99.8 % of the deploy window in every cell.** The replay rate is sustained across a tenfold journal range; were it linear in expanded state rather than in compact-event count, the rate would fall as state grew. It does not.

Three properties of the measurement deserve naming. First, the substrate cost of deployment is linear in compact-event count, not in expanded state — replaying ten times the journal takes ten times the time, at constant rate. Second, the orchestration tail is empirically negligible at every measured size; the deploy window *is* the replay window to within a fraction of one percent. Third, the régime under which this holds is the régime Paper 2 already named the canonical one: parametric verbs encoded as `ActionId` plus arguments, not literal scripts. A journal of literal scripts would replay slower in proportion to script length; this paper concerns the compact-action substrate named in Paper 2.

Apparatus The handover window can be rendered as a sequence between four roles. The leader is the live instance accepting writes; the journal is the substrate; the follower is the new instance approaching the present head; the gate is the predicate that decides which instance is authoritative at any moment.

sequenceDiagram

```

autonumber
participant L as Leader (authoritative)
participant J as Journal (substrate)
participant F as Follower (new version)
participant G as Gate

Note over F,G: Follower starts; gate closed for follower
F->>J: open from entry 0 (or checkpoint)
loop bulk replay
    J-->>F: next entry
    F-->>F: apply entry to in-memory state
end
Note over F: follower at near-head
F->>L: request pause
L->>L: stop accepting writes
loop handover tail
    J-->>F: residual entry written before pause
    F-->>F: apply
end
Note over F: follower at present head
F->>G: open for follower

```

G-->>L: close for leader
Note over L: leader released; follower authoritative

The diagram contains no operation other than reading from the journal and applying. The leader writes nothing between pause and release; the follower reads from the same artifact at the same offsets the leader was about to read had it continued. The gate’s flip is bookkeeping: it nominates which instance is authoritative; it neither produces nor consumes substrate content.

One canonical exception The equivalence developed here covers the in-process deployment of a single program. One canonical case lies outside its scope: a coordinated cut-over of a producer/consumer pair on an external message broker (typically Kafka) when the message format itself changes between versions. There the two endpoints must release together, because the substrate they share is the broker’s topic — a contract between independent programs — rather than the journal of either. The construct of this paper does not deny this case; it narrows its scope. Within a journaled program, deployment is replay; across two journaled programs that share a broker topic with a changing schema, the cut-over remains a separate concern. Paper 3, claim 8, named this exception in the discussion of in-actor continuity; it is preserved here without amendment.

To the next equivalence §5.1 has shown that bringing a new version of a program to a state in which it can serve is replay of the substrate up to the present head, and that the orchestration around that replay is residual at every measured size. The equivalence varies the position of the read cursor in time — at the present head of the same substrate. §5.2 varies the position of the substrate in space: a second instance, at another site, reconstructs the same program by reading the same sequence of entries the first instance wrote.

5.2 E2 — Replication is sharing history

§4.2 stated the equivalence: a second instance of the program, running on a different machine or in a different site, reconstructs the program by reading the same sequence of entries the first instance wrote. §5.2 develops the operational realization in two parts. First, that the bytes streamed between sites are structurally dense — the substrate’s wire form carries named operations, not serialized states, by construction. Second, that a consumer reading those bytes reaches the same program as the writer by replay alone, without recourse to a coordinator. The transport across which the bytes travel is treated as a witness — an off-the-shelf service that satisfies the substrate’s requirement on consumer-pull delivery.

The compact wire Paper 2, §2.3 (*dense journaling as reference*), established that under separability the journal entry of an invocation reduces to a reference to the parametric verb’s definition plus the arguments of the call. The verb’s

body is not present in the entry; it lives once, named, in the corpus’s vocabulary. Replication across datacenters is the transmission of those entries, not of states; the wire encodes the program as it was uttered, not as it was applied.

L2 (lab `paper05-lab2-bytes-per-event @ c68b2f4`) instruments the codec on the substrate’s wire surface, comparing the compact encoding (`ActionId` + parameters) against the literal encoding (script DSL + parameter assignments) at three tiers of verb richness, with and without an opportunistic `gzip` on the literal form.

Bytes-per-event in compact form is 36 at the trivial-arithmetic tier (68-byte script body), 36 at the branching-arithmetic tier (99-byte body), and 99.7 at the production-shaped tier (681-byte body). The literal form at the same three tiers is 121, 153, and 913.7 bytes. The ratio between literal and compact rises from **3.4× at tier 1** to **9.2× at tier 3**. Applying `gzip` to the literal form closes the gap only to $3.0\times / 3.7\times / \mathbf{3.9\times \text{ at tier 3}}$. The structural saving cannot be recovered by an opportunistic compressor, because the script body is not present in the compact form at all — it lives once in the definition record (912 bytes at tier 3, amortized to 0.91 bytes per invocation over 1 000 calls) and is referenced by a 4-byte `ActionId` thereafter.

The cross-validation against Paper 2 Lab 4 sharpens the claim. Tier 3’s compact payload is 67.7 bytes here; Paper 2 Lab 4 measured 67 bytes for the production verb that the synthetic tier-3 stand-in calibrates against. The two numbers agree within rounding. At full production scale, Paper 2 Lab 4 reports the literal-to-compact ratio reaches 20× over a thousand invocations of a real ticket-purchase verb. The 9.2× headline here is a conservative lower bound under the lab’s literal-projection; the production figure is reported in Paper 2 Lab 4.

Three properties of the measurement deserve naming. First, density is structural rather than algorithmic — compression operating after the fact cannot recover what naming achieved by construction. Second, the saving grows with verb richness; a domain whose operations are named transitions (Paper 2 §3) replicates per event at a width that does not scale with the operation’s internal complexity. Third, replication across datacenters, in this régime, is event streaming over a substrate that does not carry state by construction — the wire is the program as uttered, not as expanded.

The push-to-pull inversion The substrate is written by the producer and read by the consumer; the role of the transport between them is to hold the corpus durably and to deliver entries at the consumer’s pace. The producer does not know the consumer’s rate; the consumer’s progress is measured against the substrate’s own offsets, not against any signal from the producer. A consumer that lags absorbs the lag in its own cursor; the producer’s write path is unaffected.

This inversion is not a property of any particular delivery service. It is a property of the substrate. The producer’s journal is the source of record; any consumer

downstream is a reader of that record. An off-the-shelf webhook delivery service satisfies the consumer-pull requirement without adding any structure the substrate did not already license: it accepts an inbound POST from the producer’s local journal callback, holds it durably, and offers it to subscribers at the rate they fetch. The fact that an off-the-shelf service suffices is the relevant observation; the service is a witness to the substrate’s structural sufficiency, not an extension of it.

The shape can be rendered between five roles: the primary instance at site A, the primary’s journal, the delivery service, the consumer at site B, and the consumer’s journal.

```
sequenceDiagram
    autonumber
    participant P as Primary (site A)
    participant Ja as Journal A (substrate)
    participant T as Delivery (off-the-shelf, e.g. SVIX)
    participant C as Consumer (site B)
    participant Jb as Journal B (substrate)

    P->>Ja: append entry n
    Ja->>P: callback (entry n written)
    P->>T: post entry n
    Note over T: held durably; offered at subscriber's pace
    C->>T: fetch next
    T-->>C: entry n
    C->>Jb: append entry n
    C->>C: apply entry to in-memory state
```

The diagram contains no synchronous coupling between producer and consumer. The producer’s append-and-post is local; the consumer’s fetch-and-apply is local; the durable buffer between them is the delivery service. The substrate at A and the substrate at B contain the same entry at the same offset — the two journals are bit-equal under the measurement examined next.

The symmetric consumer §4.2 stated that replication “differs from deployment only in the site at which the same corpus is replayed.” The structural content of this claim is that the consumer at site B, running the same binary as the producer at site A, derives the same program from the corpus it reads as the producer derived from the corpus it wrote. No coordinator is required to synchronise the two sites; the substrate is the coordinator. This property removes the need for any external coordinator between sites: coordination is implicit in the shared corpus rather than enacted by a privileged runtime.

L3 (lab paper05-lab3-inproc-symmetric @ 3984c5d) measures this property at the level of Reactions — the partition primitive of Paper 3 — under two terminators, **Emit** and **MarkAsSkip** (Elide). Two instances of the same actor

binary consume the same event stream; the lab compares, at the byte level, both the callback tuples each instance’s Reactions produce and the journal segments each instance writes. Across 6 cells ($N = \{100, 500, 1\,000\} \times K = 2$ repetitions), **all 6 cells produce 0 callback byte diffs and 0 journal-segment byte diffs**. The instance at site B reconstructs the Reaction output entirely from the journal prefix it reads.

The discipline that makes this work without a coordinator is a property of the program, not of the runtime. A Reaction’s guard refers to data the program has explicitly *exposed* alongside its events — the **expose** clause established in Paper 3. Whatever a Reaction at B needs to evaluate a guard travels in the corpus the program writes; whatever the program chose not to expose, no remote evaluator can see. The substrate carries the program, and the program carries the contract on which its consumers depend.

Paper 4, §5.3 and claim 10, named the closely related property for cross-actor causation: the journal record of the act of speaking from one actor to another is what survives cross-datacenter replication, where canonical apparatuses (CDC, log shipping, message-queue replication) reconstruct state and lose the causal chain that produced it. §5.2 of the present paper completes the picture from the substrate side: the journal record is what the consumer reads, and the consumer’s Reactions derive from it without needing the producer’s runtime to be reachable.

Two caveats are worth naming, in the spirit of Capa 2 honesty. L3’s transport is in-process — a channel between two instances in the same process — and its measurement caps at $N = 1\,000$ entries (beyond which the harness becomes memory-bound). The symmetric property is structurally insensitive to the transport’s distance and to N : a Reaction’s match path is per-entry, and the journal-segment diff at $N = 1\,000$ is identically zero across all measured cells. But the lab demonstrates the property in vitro; full cross-datacenter delivery is the operational mode realised in §7.2, and the higher- N regime is bounded by the consumer’s append throughput rather than by anything the matcher does.

What §5.2 establishes, and what §5.3 develops next §5.2 has shown that replication, in the substrate’s terms, is the streaming of named operations over a wire whose density is by construction; that the transport between sites can be an off-the-shelf consumer-pull service; and that a consumer running the same binary reaches the same program as the producer by reading the corpus it shares. The equivalence varies the spatial position of the corpus while preserving replay as the means of reconstruction.

§5.3 varies the spatial position of the corpus without reconstructing the program at the destination. A consumer that reads the substrate and persists it locally — without instantiating the runtime that would replay it — holds the program in the form it was written. Backup, in this view, is replication arrested at the moment of holding rather than carried through to replay.

5.3 E3 — Backup is copying the program

§4.2 stated the equivalence: a consumer that reads entries from the substrate and persists them locally — without instantiating the runtime that would replay them — holds a copy of the program in the form in which it was written. The canonical literature on backup (§2.3) treats backup as a captured state, archived periodically by an external apparatus and replayed only on recovery. The equivalence stated here re-reads the operation: for a journaled program, backup is the program materializing in another substrate, and the destination’s choice is whether to hold the corpus or replay it.

The structural argument The equivalence follows in four steps from the substrate condition of §4.1:

1. If the journal is the program (Papers 1 and 2), the artifact whose duplication preserves the program is the journal.
2. Duplication of the journal is the streaming of named entries to a destination substrate — the program materializing there, sentence by sentence, in the order it was uttered.
3. The destination need not instantiate the runtime to receive the corpus; it need only persist what arrives. The consumer is a substrate, not a process.
4. When and if the destination chooses to replay, the corpus reconstructs the program. The choice between *holding* and *replaying* is independent of the act of materializing.

Three symmetries Backup, in this view, is positioned by its relation to the three other equivalences of §4.2. The position is fixed by what the destination commits to, not by any new operation.

- **Backup and replication.** Replication (§5.2) is the destination streaming the corpus and replaying it; backup is the destination streaming the corpus and holding it. The same protocol governs both. Replication is backup carried through to replay; backup is replication arrested before replay.
- **Backup and deployment.** Deployment (§5.1) is replay at a different time of the same site, ending with the new instance admitted as authoritative; backup is replay-eligible material held at a different site, with no instance ever admitted as authoritative there. The same protocol reads the corpus forward; what differs is the gate flip at the end — present in deployment, absent in backup.
- **Backup and offline operation.** Offline operation (§5.4) is the consumer reading the corpus with a delay between write and read but eventually catching up and serving; backup is the consumer reading the corpus and never serving. The same protocol delivers entries; what differs is whether the consumer ever takes a turn at serving requests.

The three symmetries are not corollaries discovered after the fact. They follow from the substrate condition directly: each of the four equivalences is read off the

substrate by varying one of two axes — when the read happens, and what the consumer commits to once it has read — and the four exhaust the combinations. §4.3 named this arrangement.

What the substrate measures L4 (lab paper05-lab4-passive-consumer @ a21a655) instruments the materialize cycle on three storage backends under the compact-action régime of L1. The dataset spans 3 backends $\times$ 3 journal sizes $\times$ 2 protocol layers $\times$ K = 2 measurement repetitions = 36 cells, plus 3 catch-up cells.

Under régime N = 100 000 compact entries, the destination journal converges at **2 419 events/sec on FileSystem, 1 004 events/sec on MySQL,** and **764 events/sec on SQL Server (SQL Edge stand-in)**. The wire round-trip itself accounts for **less than 0.6 % of the cycle** at every cell; the substrate cost is the destination’s append rate, not the operation. A replica actor instantiated over the destination journal — a verification step, not part of the backup operation — reaches the same in-memory state as the primary in **18 of 18 measured cells across the three backends and three journal sizes**.

The independence from backend is the empirically significant property here. The equivalence is not a feature of any storage technology; it holds wherever the journal can be appended to.

The destination as cursor The destination’s three reading regimes — near the head of the substrate, paused after an interruption, or newly instantiated after the primary has run for some time — share a single protocol. Each is a cursor on the substrate, read forward from its present position to the present head. Catch-up after a retention loss is not a separate mode; it is the same read, started further back. L4’s catch-up cells confirm this: the catch-up throughput matches the steady-state throughput within stochastic variation on each backend. The substrate does not contain a distinction between *current* and *recovering*; it contains a sequence of entries, and each consumer holds a cursor over it.

Caveats Two caveats are worth naming, in the spirit of Capa 2 honesty. First, L4’s transport between primary and destination is in-process — a local proxy, not a real cross-datacenter wrapper; the network latency $\times$ number of round-trips a cross-datacenter deployment adds is bounded but not measured here. Second, L4 measures the *passive* half of the equivalence — the destination consuming the corpus — not the act of promoting a destination to authoritative status; that promotion is the E1 case §5.1 already developed, and is structurally available to any destination that holds the corpus to a chosen EntryId.

The operational realisation of E3 — the modes under which a destination starts, how the program declares which entries materialize where, the transport that

carries them, and the dimensions along which the construct scales (heterogeneous backends, multi-destination fanout, per-datacenter topology) — is developed in §7.3.

To the next equivalence §5.3 has shown that backup is the program materializing in another substrate; that backup is positioned by its relation to the three other equivalences along two structural axes — when the read happens, and what the consumer commits to once it has read; and that the corpus convergence rate on heterogeneous backends is independent of the construct. The equivalence varies the consumer’s commitments while preserving replay as the means by which the program is reconstructed if and when needed.

§5.4 varies a different axis: the delay between when the substrate is written and when it is read. Offline operation, in this view, is replay after a gap — the consumer reads the same corpus the producer wrote, but the read trails the write by an arbitrary interval.

5.4 E4 — Offline operation is delayed replay

§4.2 stated the equivalence: a consumer that is unreachable when entries are written, and that catches up by reading them later from a persistent buffer or directly from the source, applies the same replay to the same sequence as a consumer that received the entries in real time. The canonical literature on offline operation (§2.4) treats the buffer as an apparatus added at the system’s boundary — a message queue, a durable offset, a convergence protocol — whose function is to absorb the period during which the system cannot reach its counterpart. In the regime described here, no such apparatus exists: the substrate itself is the durable buffer, and offline operation reduces mechanically to cursor advancement over the same corpus.

If the producer writes to a substrate that is local — local to the producer’s process, locally durable — then there is no point at which the producer needs to know whether any consumer is reachable. The producer’s write is to its own substrate. A consumer reading from that substrate, whether it reads in real time or after an interruption, holds a cursor over the same corpus; when the cursor advances, the consumer applies the entries between the old position and the new one. The buffer of the canonical literature is therefore not added; it is the substrate itself.

Offline as the limit of replication §5.2 developed replication as the consumer reading the corpus the producer wrote, at a different site, with the transport between them holding entries durably until the consumer fetches them. §5.4 is the limit case: the consumer’s read trails the write not by transport latency but by an arbitrary interval — minutes, hours, or longer. The protocol is identical. The act of catching up is the act of advancing the cursor over the entries that accumulated in the meantime.

The same mechanism applies in the reverse direction. A producer whose canonical remote store is unreachable continues writing to its local substrate; the remote store, when reachable again, is a consumer that fell behind. The role of *producer* and *downstream store* is asymmetric only in steady-state framing; structurally, both are cursors over the same corpus.

What the substrate measures L5 (lab paper05-lab5-offline @ d5cb906) measures the offline equivalence by configuring an actor’s substrate as a local persistent journal with an asynchronous flush to a remote canonical backend (MySQL and Azure SQL Edge). The remote backend is then partitioned for a measured interval (~5 s of write volume); the partition is resolved; the backlog drains.

Three properties of the measurement deserve naming.

1. **Decoupling of write latency from the remote.** Under online operation, the primary appends at **517 μ s p50 against MySQL** and **455 μ s p50 against SQL Edge** — **7.29 $\times$ and 2.92 $\times$ faster** than the unbuffered direct path. The producer’s lock release time is bounded by the local substrate’s append, not by the remote’s commit latency.
2. **Partition latency is not worse than online latency.** During the partition, the buffered primary’s per-append p50 is 338 μ s on MySQL and 381 μ s on SQL Edge — the same régime as online; the write lock never traverses the network. The substrate’s local persistence is what makes this hold; the absence or presence of the remote does not enter the write path.
3. **Catch-up is linear in the backlog.** On reconnect, the accumulated entries drain at the remote backend’s steady-state ingest rate — 280 events/sec on MySQL, 420 events/sec on SQL Edge — with **zero events lost** across the measured cells. Catch-up is not a separate mode; it is the remote’s cursor advancing from its last position to the present head.

Two caveats are worth naming, in the spirit of Capa 2 honesty. First, L5 partitions the remote by stopping its container — a realistic partition mode but not the only one; half-open connections without container death would exercise the connection-timeout path differently and are not measured here. Second, the partition-phase latency being at or below the online phase is partly an artifact of single-host scheduling — the asynchronous flush thread is idle during partition, freeing CPU for the writer; in production with separate threads on separate cores, the artifact shrinks, but the structural property — the write path is independent of the remote’s reachability — holds either way.

The operational realisation of E4 — the configuration that enables the local substrate as the primary’s source of truth, the asynchronous component that flushes to the canonical backend, and the catch-up path on reconnect — is developed in §7 alongside the other operational details.

To the next equivalence §5.4 has shown that offline operation is replay after a gap, and that the gap is absorbed by the substrate’s own local persistence rather than by a separate apparatus. The four equivalences of §4.2 are now developed.

§5.5 turns to the cost that underlies all four: replay against a local substrate is cheap because append against a local substrate is cheap. The latency budget that supports the four equivalences is examined as a single property of the substrate, not as a property of any one operation.

5.5 The latency budget

The four equivalences of §4.2 reduce four classical operational disciplines to a single primitive: replay. Replay is the act of reading the substrate forward from a cursor and applying each entry. Its cost is the cost of reading and applying entries. For a substrate that is read often and written often, the cost of *writing* entries — appending to the journal — is the cost basis that supports the family of replay-based operations. §5.5 examines that basis.

The structural claim that underlies E1–E4 is this: every replay-based operation reduces to advancing a cursor over N entries, and therefore inherits the substrate’s per-entry append latency as its dominant cost term. The actor’s apply cost is identical across all regimes because it operates on the same in-memory state. The only term that varies between regimes is the cost of appending entries to the substrate.

The relevant comparison is between two regimes for the substrate’s persistence. Under the regime this paper develops, the substrate is a local journal whose persistence is one *fsync*-bounded append per entry. Under the canonical regime (§2), the substrate’s role is played by an RDBMS whose persistence is one commit-bounded *INSERT* per entry. The latency gap between the two regimes propagates linearly to the cost of deployment, replication, backup, and offline catch-up. The four operational disciplines therefore share a single cost basis: the substrate’s append latency.

What the substrate measures L6 (lab paper05-lab6-latency-budget @ 86905dc) measures per-entry append latency on three local backends under a one-event = one-durable-commit régime: a local FileSystem journal, a co-located SQL Server (Azure SQL Edge stand-in), and a co-located MySQL. The dataset spans 3 backends $\times$ $K = 5$ runs $\times$ $N = 100\,000$ entries per backend = 1.5 million samples, with the first 1 000 appends per cell discarded as warm-up.

Backend	Append p50 (μ s)	Append p95 (μ s)	Append p99 (μ s)	Ratio to local FS (p50)
Local FileSystem journal	347	581	1 919	1.0 $\times$

Backend	Append p50 (μ s)	Append p95 (μ s)	Append p99 (μ s)	Ratio to local FS (p50)
Co-located SQL Server	1 126	1 891	2 580	3.24 $\times$
Co-located MySQL	982	1 498	2 021	2.83 $\times$

Durability is parameterised identically across the three backends: the FileSystem backend invokes a per-entry flush-to-disk; SQL Server runs with default per-commit log flush; MySQL runs with `innodb_flush_log_at_trx_commit=1` and `sync_binlog=1`. The measurement therefore compares the same physical durability guarantee across all three — one entry, one durable commit.

The journal-append regime favours the substrate by $3\times$ over both RDBMS engines at the median, and the same ordering holds at the long tail (p95 and p99). The advantage is *structural*: both RDBMS engines land in the same $3\times$ band, so the gap is a property of the substrate’s append being one *fsync* per record versus the RDBMS being one *fsync* per redo-log entry plus the protocol overhead of a transaction.

The corollary to E1–E4 follows. Every replay-based operation costs $3\times$ less per entry against the local-journal substrate than against a co-located RDBMS in the durable-commit regime. Deployment at N entries (E1, §5.1), replication at N entries (E2, §5.2), backup ingestion at N entries (E3, §5.3), and catch-up after a partition at N entries (E4, §5.4) inherit the same factor.

Caveats Two caveats are worth naming, in the spirit of Capa 2 honesty.

First, the comparison is *conservative against the substrate*. The RDBMS engines in L6 run on ephemeral container storage — MySQL on a tmpfs mount, SQL Edge on a Docker named volume — whereas the local journal goes through the host’s NVMe controller. On a production RDBMS deployment with real disk, the RDBMS side would slow further; the $3\times$ ratio reported here is therefore a lower bound, not an upper one. Adding network latency for a remote RDBMS — a typical production topology — adds a flat additive term per commit, pushing the ratio further still.

Second, the comparison can be narrowed by relaxing durability on the RDBMS side. Group commit, batched inserts, and asynchronous-durability modes — SQL Server’s `DELAYED_DURABILITY` and MySQL’s `innodb_flush_log_at_trx_commit=2` — all close part of the gap by trading durability for throughput. These are honest engineering choices; L6 measures the durable-commit baseline because the substrate’s local-journal regime keeps full durability and the apples-to-apples comparison demands the same of the RDBMS side. A reader who deploys an RDBMS-backed substrate with relaxed

durability will see a smaller gap; the structural claim — that replay cost is bounded by append latency — does not change, only the constant.

To the next sub-section §5.5 has shown that the cost basis of the four equivalences is one quantity — the substrate’s per-entry append latency — and that against a co-located RDBMS in the durable-commit regime, the local-journal substrate runs at $3\times$ lower latency per entry. The result is structural rather than engine-specific: two RDBMS engines, two implementations, the same ratio band. §5.6 closes §5 by naming two further consequences of the substrate — observability and debug — that fall out of the same property without requiring separate apparatuses.

5.6 Forensic operations as substrate consequences

Two further operations follow from the substrate without requiring separate apparatuses. The substrate is a total, ordered record of every named operation performed by the program. Conventional observability stacks reconstruct this record indirectly by correlating logs, metrics, and traces emitted outside the program. Under the substrate condition, the record already exists in primary form; any observability layer becomes a reader of the substrate rather than a reconstructor of program history. Sagas do not appear as patterns to be inferred post-hoc from logs; they are named at write time by the program’s partition primitive (Paper 3) and therefore exist in the substrate as first-class entries.

Replay is deterministic: the same corpus, applied to the same binary, produces the same in-memory state at the same cursor position. A runtime issue traceable to an entry is therefore reproducible by replaying the substrate to that entry’s position; a conditional breakpoint over the entry’s identifier is trivial because the identifier is a position on the cursor, and the cursor advances one entry at a time. Debugging reduces from forensic analysis over logs to advancing the cursor to a chosen position in the substrate. Both observations exist on the same axis as §5.1–§5.5: they require no additional mechanism beyond the substrate itself. §6 turns to the cost in operational complexity that the canonical disciplines pay to achieve, by separate apparatuses, what the substrate already provides.

6. Why existing approaches duplicate work the substrate already does

The four disciplines surveyed in §2 are not unrecognised by their respective traditions. Each has developed, over decades, a refined apparatus to address the operational concern it names — blue-green and rolling-update for deployment, change-data-capture and log shipping for replication, snapshot agents and point-in-time recovery for backup, message-queue offsets and convergence protocols for offline operation. The maturity of these apparatuses is not in question. What this section examines is whether any of them dissolves the operational concern

it addresses, or whether each reconstructs, from outside the program, work that the substrate condition of §4.1 already does.

This section is not a critique of existing practices but a structural comparison between what those practices reconstruct and what the substrate condition provides by construction. The four sub-sections below examine one canonical apparatus per discipline. The pattern of each examination is the same: name what the apparatus does, identify the structural property it relies on, and observe that the substrate condition provides that property by construction. The point is not that the apparatuses are wrong but that they are responses to the absence of the substrate condition. Once the condition holds, the work each apparatus performs is already done by the program’s own act of writing the journal.

6.1 Blue-green vs substrate

Blue-green deployment [Fowler 2010] provisions two parallel production environments and switches traffic between them at release time. The apparatus has three components: a duplicate environment, a router that decides which environment is authoritative at any moment, and a procedure for bringing the standby environment to the state from which it can accept traffic. The first two are operational infrastructure; the third — bringing the standby to a serving state — is what the apparatus does that the substrate condition recasts.

Under the substrate condition (§4.1), the act of bringing a process to a state in which it can serve is the act of replaying the journal up to the present head (E1, §4.2). The new version’s instantiation differs from the old version’s continued operation only in the position of its read cursor. The duplicate environment is therefore not a parallel system to which state must be propagated; it is a process whose cursor on the same corpus has not yet reached the present head. The replay that the apparatus performs implicitly — by re-running migration scripts, by exporting and re-importing state, by re-priming caches — is performed explicitly by the substrate’s reading discipline.

The structural redundancy is not that blue-green is wrong; it is that blue-green’s hardest step, the act of synchronising the standby with the live state, is replay reconstructed manually. The reconstruction is manual because the live state, in the canonical regime, is not the journal of the program; it is a database whose entries record what is, not what was said. Under the substrate condition the manual reconstruction has nothing to do, because the corpus from which the standby reads is the same corpus the live instance writes. The router and the duplicate environment remain; what dissolves is the apparatus that bridges the two.

6.2 Change-data-capture vs event streaming

Change-data-capture [Debezium project; Maxwell project] reads the binary log of a relational store and re-emits each row-level change as an event on an ex-

ternal transport. The apparatus exists because the originating system does not emit events; it emits state changes, and CDC observes those changes and translates them back into the event form that downstream consumers require. The translation step is the apparatus.

The translation is informationally lossy in one direction and inferential in the other. State changes do not carry the operation that produced them; CDC infers, from a sequence of column-level deltas, what the application program intended. The inference is necessarily heuristic — two distinct programmatic operations can produce identical column-level deltas, and the CDC consumer cannot recover the distinction. The apparatus is therefore not only a transport; it is a reconstruction layer that approximates events from state, with the loss inherent in that approximation.

Under the substrate condition the reconstruction has no work to do. The originating system emits events — they are the entries of the journal, written in the language of the program (Paper 1, anti-porosity) and reduced to compact named operations (Paper 2, separability). The wire across which replication travels carries those entries directly (§5.2); no inference from state, no column-level delta extraction, no schema mapping enters the act. The CDC apparatus is structurally duplicative: the substrate emits exactly the artifact CDC reconstructs — minus the reconstruction loss, and minus the apparatus that produces it.

6.3 Snapshot backup vs passive consumer

Snapshot backup [WAFL; PostgreSQL PITR; Veeam] captures the state of a data store at a chosen moment and writes that capture to an archival medium. Recovery instantiates the captured state and resumes operation from it. The apparatus has two components: the snapshot mechanism that produces the capture, and the recovery procedure that consumes it. Both operate on state — the captured artifact is a frozen image of what the program had stored, not a record of what the program had done.

The structural property the apparatus relies on is that the program’s state, if captured atomically at a moment, suffices to characterise the program at that moment. The property holds for programs that are reducible to their state. It does not hold for programs that are reducible to their history: a snapshot of the state at moment t does not record the operations that produced it, only their cumulative effect, and recovery from the snapshot loses the operational record.

Under the substrate condition the program is not reducible to its state; it is constituted by the sequence of operations in its journal (§4.1). A consumer that holds the corpus — without instantiating the runtime that would replay it — holds the program in the form in which it was written (E3, §4.2; §5.3). The choice between holding and replaying is a property of the destination, not of the captured artifact. The snapshot apparatus produces a state image and discards the operational record; the substrate emits the operational record continuously

and lets the destination decide whether and when to replay.

The redundancy is structural in two directions. First, the snapshot apparatus is unnecessary: the substrate is already the program, copied continuously. Second, the snapshot apparatus is also insufficient: it captures state, which is less than the program — recovery from a snapshot reconstructs *where* the program was, not *what it did to get there*. The passive consumer of §5.3 holds the program, and any moment in the program’s history is replayable from the corpus the consumer holds.

6.4 Message queue buffer vs persistent local journal

The message-queue pattern places a durable buffer between a producer and a consumer that may not be reachable in real time. RabbitMQ’s dead-letter exchanges, Kafka’s consumer offsets, and the eventually-consistent stores derived from Dynamo [DeCandia et al. 2007] each place an apparatus at the boundary between the system and its counterpart. The apparatus is external to the producer; the producer writes to its store, and a separate mechanism — the queue, the broker, the convergence protocol — handles the case in which the consumer is unreachable.

The structural property the apparatus relies on is that the producer’s store and the consumer’s store are different artifacts, with a transport between them that must be made durable. Under that structural premise, the queue is the right apparatus: it absorbs the period during which the consumer cannot reach the producer, and it persists the messages that would otherwise be lost.

Under the substrate condition the producer’s store and the consumer’s store are not different artifacts at the level of representation. The producer writes to its substrate; the consumer reads from a copy of the same substrate, or from the substrate directly. The substrate is itself the durable artifact, and offline operation reduces to the consumer’s cursor trailing the producer’s write head by an arbitrary interval (E4, §4.2; §5.4). No queue is added between the two parties because the substrate is already the queue — it persists every operation in the order it was uttered, and a consumer that has been unreachable for an arbitrary period catches up by reading the entries that accumulated.

The redundancy here is the most structural of the four. Where the queue, the broker, and the convergence protocol exist as boundary infrastructure, the substrate is itself the boundary — it is what holds the program between writer and reader, between connected and disconnected, between now and later. The infrastructure that the canonical literature places between two stores is, under the substrate condition, the property of the single store both parties share.

6.5 The substrate match table

The four examinations of §6.1–§6.4 share a structure. Each canonical pattern relies on a property the substrate condition supplies by construction; each ap-

paratus performs work that the journal, written as the program, already does. The pattern is summarised in the table below, in the form of Paper 4 §6.4 [Paper 4 §6.4]: a single question — *does the apparatus address a property the substrate already provides?* — answered for each pattern.

Pattern	Property the apparatus relies on	Does the substrate condition already provide it?
Blue-green deployment	The standby must be brought to the same state as the live instance before traffic switches	Yes — replay of the same corpus to the present head (E1, §5.1) brings the standby to that state by construction; no external state-propagation step is required
Change-data-capture	The originating store emits state changes; events must be reconstructed from them	Yes — the substrate emits the operations themselves (Papers 1 and 2; §5.2); no reconstruction step is required and no inferential loss is incurred
Snapshot backup	The program is characterised by its state at a moment; recovery instantiates that state	Yes — the program is constituted by the sequence of operations in the journal (§4.1, §5.3); the corpus held by a passive consumer is the program, not a state capture of it
Message queue buffer	A durable apparatus is required between the producer’s store and the consumer’s store	Yes — the substrate is the durable artifact both parties share (§5.4); a consumer reading later than the producer wrote reduces to cursor advancement over the same corpus

The four “Yes”s share a structure. In every case, an apparatus that the canon developed to bridge an absence of the substrate condition becomes structurally

subsumed once the condition holds. The pattern remains usable — none of the four is wrong as engineering — but the work it performs duplicates work the substrate already does. The maturity, sophistication, and widespread adoption of the four patterns are not evidence that the substrate condition is unnecessary. They are evidence that the absence of the substrate condition is costly enough to justify four separate apparatuses to compensate for it, each developed independently by a different tradition.

The diagnosis of §2–§6 is now complete. §7 turns from the construct to the instantiation: a system in production whose runtime already satisfies the substrate condition, whose four operational disciplines fall out of the substrate without dedicated apparatus, and whose existence is the evidence that the construct names a realisable property and not only a conceptual one.

7. Instantiation: realization in Puppeteer

The construct of §§3–6 makes claims about a class of systems characterised by the substrate condition. The present section exhibits one such system. The voice from this point forward is concentrated and authorial: the runtime described below is *Puppeteer*, the framework the present author maintains, and the operational decisions named are decisions taken in deployments that run today. The framework is presented here as an existence proof for the construct rather than as the subject of the paper; the four equivalences hold in any runtime that meets the substrate condition, of which Puppeteer is one.

7.0 Origins

Paper 3, §6 introduces the runtime decomposition this section relies on: *Performance* is the local hosting of a single actor over a configured storage, *Theater* is the host for one or many Performances on a node, *Ensemble* is a pool of actors with routing, and *StageManager* is the peer-to-peer replicated state machine that coordinates topology across nodes. The full vocabulary is presented in Paper 3 §6 and is not re-introduced here. §7 names only the runtime surfaces on which the four equivalences of §4.2 land: the *aliveGate* (E1, §7.1), the per-event push callback wired to a webhook delivery service (E2, §7.2), and the *Materialize* construct that declares which entries materialise in which destinations (E3, §7.3). The offline equivalence (E4) lands on the same write path the local-journal régime of §5.4 already exhibited, and is referenced rather than re-developed here.

7.1 Theater.aliveGate: the red-black mechanism

The handover window of §5.1 is realised in the runtime by a single boolean predicate: the *aliveGate*, a manual-reset event held on every **Performance** instance. The gate is the locus of the deployment apparatus, and its life-cycle is

the operational realisation of the structural intervals named in §5.1.

A **Performance** is started in one of two modes. A primary starts with **Start()** (default **asFollower: false**): the runtime hydrates the actor from its substrate, raises the **OnFirstHydration** and **OnHydrated** callbacks, and sets the gate. A follower starts with **Start(asFollower: true)**: the runtime hydrates the actor from the same substrate, but the gate remains reset and the public **IsAlive** predicate reports **false**. External callers — request dispatchers, write coordinators — block on **WaitUntilAlive** until the gate is set.

The handover is a single sequence over the gate. The follower invokes **LockWhileNotSynchronized()**, which pauses writes on the live primary and returns the **EntryId** at which the pause occurred. The follower then catches up over the residual tail with **CatchUpFromJournal(targetEntryId)**, which replays pending events under the actor’s write lock until the follower’s cursor reaches the targeted **EntryId**. **UnlockAndRunAlive()** flips the follower’s gate and releases the primary. From the perspective of any external caller, the system transitions from “primary alive” to “follower alive” in a single atomic moment — the gate flip — preceded by replay of the substrate and nothing else.

The structural content of the apparatus is therefore minimal. The gate is book-keeping over the substrate’s read cursor; the runtime contains no separate deployment subsystem, no state-extraction layer, and no schema-mediated handoff. The orchestration tail measured in §5.1 — below 30 μ s at every measured N — is the cost of the gate flip and the residual-tail replay, not of any apparatus. Appendix A lists the exact source locations of each named surface.

7.2 Event streaming: push-to-pull inversion

The substrate’s push-to-pull inversion (§5.2) is realised in Puppeteer by composing three primitives that already live in the runtime. None of them was introduced for §7.2; each existed for its own structural reason, and together they satisfy the consumer-pull requirement.

First, the journal exposes a per-record callback. **Diary.OnRecordWritten** is invoked by the storage layer for every entry written, with the entry’s **entryId** and its raw byte representation. The setter accepts a single subscriber; the companion **AddRecordWrittenCallback** chains additional subscribers without disturbing the existing one, so that multiple downstream consumers can attach to the same primary without the primary knowing how many.

Second, the partition primitive of Paper 3 exposes a *Cue* mode for Reactions. A **Cue()** Reaction runs continuously: it batches over the substrate from the current cursor to the present head and then enters *push mode*, subscribing to the same **OnRecordWritten** callback above. The push loop applies each entry as it arrives, matches it against the Reaction’s pattern, and dispatches the Reaction’s terminator if the pattern matches.

Third, the substrate’s externalisation point is the terminator of a **Cue** Reaction. The terminator’s body — DSL the program writes — is the place at which the runtime delegates to whatever external delivery service the deployment has elected. We observe that an off-the-shelf webhook delivery service (SVIX, in the deployments referenced here; equivalent products would suffice) satisfies the substrate’s consumer-pull requirement: the Reaction’s terminator posts the entry to the service from a per-entry callback; the service holds the entry durably; downstream subscribers fetch from the service at their own pace.

No code in **Puppeteer/** or **Choreography/** references SVIX. The runtime exposes the callback and the partition primitive; the wiring to a particular webhook service is a property of the deployment, not of the framework. The substrate’s claim is that any service satisfying the consumer-pull contract is sufficient; the off-the-shelf adoption is the empirical witness to that sufficiency.

7.3 Passive consumer: replicate without an actor running

The passive consumer of §5.3 is the operational realisation that, in earlier drafts of this paper, was named as an honest gap — a property derivable from the substrate condition but not yet exhibited as a system in production. The gap closed on the day this section was drafted, by composition of patterns the runtime already had. The composition has since been formalised under the construct *Materialize*. The remainder of this sub-section walks the composition brick by brick, in the spirit of the andragogical commitment of Paper 2: a reader who has followed §§4–6 is at risk of dismissing E3 as “stated, not exhibited”; the present sub-section makes the assembly visible so that the dismissal has nothing to dismiss.

The instantiation has four bricks. The reader is asked to hold them separately and observe how they compose.

Brick 1 — Same binary, alternate startup mode. A destination is a process running the same actor binary as the primary, started in an alternate mode. The mode admits no external requests: the **aliveGate** of §7.1 is not opened for the destination’s lifetime. Two such modes exist. **Performance.Start(asFollower: true)** starts the actor as a *follower* — a process whose **Job()** Reactions run against a shared substrate; the follower is a worker, not a backup, and its Reactions contribute markers back to shared auxiliary tables. The second mode, **/materialize**, runs no Reactions at all — it consumes pre-computed markers from the primary via the wire protocol described under Brick 3. The mode is selected by the host process at startup; the runtime is the same in either case.

Brick 2 — Per-event marker in the DSL. The program declares which entries materialise in which destinations through a marker on the Reaction’s metadata plane:

```
actor.Reactions.DefineReaction("...")
```

```

        .Job().Company().ReadForward()
        .Seek("...")
        .OnMatch("...")
        .Metadata.Materialize("DC-B");

```

`Metadata.Materialize("DC-B")` records, for every entry that matches the pattern, a row in the `EventMaterialization` table naming the destination. The row is the program’s declaration that this entry is part of the corpus the named destination is to receive. Reading the program reveals which entries are intended for which destinations, in the same DSL the program is written in — the construct is, in the sense of Paper 1, an instance of the homoiconic recording the substrate condition requires.

Brick 3 — The wire protocol between primary and destination.

The destination, when it wakes up, asks the primary for the entries it has not yet seen. The exchange is realised by four wire verbs on the primary’s `actor.Materialization` sub-namespace. `ReadRecordsAfter(destination, fromEntryId)` returns the raw substrate records the destination is missing — Capa 1, the program as the primary wrote it. `ConfirmUntil(destination, entryId)` records the destination’s acknowledgment on the primary side as a Max-monotonic watermark — the primary now knows the destination has the corpus up to that `EntryId`. `ReadReactions(destination)` and `ReadElidedRange(destination, from, to)` return Capa 2 derived state — the Reaction registry’s atomic snapshot and the elision markers the primary computed in the same range — so that a destination electing to replay can reach the same in-memory state the primary reached without re-running the Reactions itself.

The destination side is the fluent client `MaterializeMirror`. The contract is two-layered:

```

mirror.Sync();                // Capa 1 - orchestrates (a) + (b).
mirror.AsProgramMirror().Sync(); // Capa 2 - orchestrates (a) + (c) + (d) + (b).

```

`Sync()` fetches records and confirms the watermark; `AsProgramMirror().Sync()` additionally fetches the Reactions snapshot and elision markers, so that the destination’s program state is reconstructible without instantiating the Reaction matcher. The destination decides what to do with the result — a passive backup persists the records and discards Capa 2; a replicated read-replica persists both and applies the elision; a snapshot-time mirror runs Capa 1 only.

Brick 4 — Recovery primitive. When a destination has been offline for an arbitrary period — or when an intermediate delivery service has exhausted its retention — the destination resumes by issuing `Sync()` against its current watermark. The primary serves the missing entries from its substrate; the destination’s watermark advances; the round-trip is the same as steady-state. There is no separate recovery mode in the runtime, because there is no separate steady-state mode either: every fetch is `Sync()` against the watermark,

and every catch-up is `Sync()` over a wider range. The recovery primitive is the steady-state primitive.

The four bricks compose into a single operation. The program declares which entries materialise in which destinations (Brick 2). A destination process — running the same binary in `/materialize` mode (Brick 1) — fetches records from the primary’s Materialization surface (Brick 3) at its own pace, confirms what it has received, and catches up after any interruption by the same call (Brick 4). The runtime contains no replicator, no separate backup subsystem, and no synchronisation layer between primary and destination beyond the four wire verbs and the per-Reaction marker. The operation falls out of the substrate; the construct names the operation that the program already declares.

Three dimensions of the construct deserve naming, because each is structurally present without requiring a separate apparatus.

Heterogeneous backends. `Diary` selects its storage implementation polymorphically from (`DatabaseType`, `connectionString`) at construction time. A primary running on the local `FileSystem` can materialise to a destination running on MySQL or SQL Server, and the wire protocol carries the same raw records across the heterogeneity — `ReadRecordsAfter` returns `Capa 1` records regardless of which backend the primary stores them in. The four wire verbs are implemented on each of the four backends with the same contract; L4 (§5.3) exhibits this independence with **18 of 18 cells reaching equal in-memory state across three backends and three journal sizes**.

Multi-destination fanout. `AddRecordWrittenCallback` chains additional subscribers without displacing the existing one. Multiple destinations may register against the same primary; the primary’s write path is unaware of how many. Each destination holds its own watermark on the primary side; each progresses at its own rate; the primary serves them independently. Multi-backup of one actor — and the per-destination retention policies that go with it — falls out of the chain primitive.

Per-datacenter topology. A datacenter is an operational placement of `Performance` instances; it is not a construct the actor knows about. A primary at datacenter A and a destination at datacenter B is a topology decision realised by the operator: a local SVIX at each site for the Cue feed of §7.2, and `/materialize` processes at each site whose mirror clients fetch from the local SVIX or directly from the primary. The substrate’s locality property (the journal is the actor’s, not the system’s; cross-ref §6.2 of Paper 3) extends laterally: each datacenter’s destinations are wired to the local instance of the delivery service, and cross-site bandwidth is consumed only by the SVIX-to-SVIX replication the off-the-shelf service performs.

A gap is named honestly in the spirit of `Capa 2`. The `Diary.OnRecordWritten` setter is wired today for primary `FileSystem` and for the buffered régime; for primary MySQL or SQL Server the setter is a silent no-op. The canonical case of the substrate (a `FileSystem`-primary deployment) is unaffected, and the §7.2

push-to-pull path runs against the FileSystem callback as designed. A future deployment with primary MySQL would require the callback to be wired in the MySQL storage path — a localised patch, not a structural change.

7.4 Numbers

The operational régime of the four bricks above runs under the latency budget §5.5 already developed. The substrate cost of materialisation is the cost of appending the program at the primary (which §5.5 measures) plus the cost of appending at the destination (which §5.3's L4 measures). No new numbers are introduced here; the relevant ones live in §5.1 (deployment window), §5.2 (wire density), §5.3 (destination convergence), §5.4 (offline catch-up), and §5.5 (per-entry append latency). Together they characterise the régime in which the four bricks above operate; together they materialise the construct in production-grade numbers.

8. Relation to previous work in this paper series

The substrate condition stated in §4.1 — that the journal is homoiconic, dense, and made of compact named operations — is not asserted ex nihilo by the present paper. Each of its components is the conclusion of prior structural analysis in this series, and each entered §4.1 as a precondition rather than as a claim of this paper. Without those preconditions the four equivalences of §4.2 would not follow; with them, the four equivalences are corollaries of the substrate's properties under variations of when and where the read happens. §8 makes the chain of dependence explicit.

Paper 1 introduces *anti-porosity* as a design principle for the boundary between domain code and persisted form. The journal records what the program said, not the data structures the program manipulated; entries are sentences in the same language the program is written in, and every effect that crosses the boundary is named in that language rather than summarised as a state delta. Anti-porosity is the precondition under which the journal can be the program at all: a porous journal records states whose operational origin is lost, and the substrate condition reduces to recording a derivative of the program rather than the program itself. §4.1 takes anti-porosity as given; without it, E2 (replication as sharing history) and E3 (backup as copying the program) collapse into the canonical regimes of §2.

Paper 2 introduces *separability* as the structural property that makes the journal entry of a parametric verb a compact reference plus its arguments. The verb's body is not present in the entry; it lives once, named, in the corpus's vocabulary. Separability is the precondition under which the wire of §5.2 carries operations at a density that does not scale with the operation's internal complexity, and it is the precondition under which the latency budget of §5.5 holds — replay is linear in compact-entry count, not in expanded state, only because separability has

reduced the entry to its reference form. §5.5’s three-times advantage against co-located RDBMS engines is, in part, the operational expression of the structural property Paper 2 names.

Paper 3 introduces *the partition* between immediate work, done at the actor’s writer-lock cursor, and deferred work, done by Reactions that read the journal forward and produce their own derived entries. Paper 3 §6 establishes the runtime decomposition — *Performance, Theater, Ensemble* — that frames the operational realisation of §7. The mechanism that admits a new instance as authoritative (§5.1’s gate flip) is named in Paper 3 claim 8 as a property derivable from the partition; the same paper names the cross-actor extension as a forward reference closed by Paper 4. §5.2’s symmetric-consumer property — that two instances of the same binary, reading the same corpus, produce identical Reaction output (L3) — depends on the **expose** discipline Paper 3 §5 establishes: a Reaction’s guard reads only data the program has explicitly exposed, so whatever the remote consumer needs to evaluate the guard travels in the corpus the program writes.

Paper 4 introduces *tell* as the primitive under which cross-actor causation is recorded as program in each participant’s local journal. Paper 4 claim 10 names cross-datacenter replication as the canonical case in which tell’s program-level recording survives where the conventional apparatus (CDC, log shipping) reconstructs state and loses the causal chain. §5.2 of the present paper completes the picture from the substrate side: the journal record is what the consumer reads, and the consumer reconstructs not only the local actor’s state but, when tell entries are present, the cross-actor causal chain those entries record. The substrate condition makes Paper 4’s cross-DC property hold without an external mechanism that observes the producer’s runtime.

The dependence is summarised in the table below.

Paper	What it establishes	What §4.1 takes as given	Where it is load-bearing in this paper
Paper 1	Anti-porosity: the journal records operations, not state	The journal is homoiconic and dense	E2 wire density (§5.2); E3 the corpus is the program (§5.3); §6.3 snapshot is informationally less

Paper	What it establishes	What §4.1 takes as given	Where it is load-bearing in this paper
Paper 2	Separability: the entry of a parametric verb is <code>ActionId</code> + arguments	The journal entry is compact, named, parameter-referenced	Wire density at production scale (§5.2; L2); replay rate linear in compact count, not state (§5.1; L1); latency budget (§5.5; L6)
Paper 3	Partition: immediate vs deferred; Reactions; runtime decomposition (<i>Performance</i> , <i>Theater</i> , <i>Ensemble</i>)	Reactions and the partition primitive; the runtime apparatus referenced in §7	Gate flip apparatus (§5.1; §7.1); symmetric consumer via <i>expose</i> (§5.2; L3); apparatus vocabulary (§7.0)
Paper 4	<i>tell</i> primitive: cross-actor causation as program in sender’s journal	The cross-actor recording the substrate carries across sites	Cross-DC causal chain travels with replication (§5.2; cross-ref claim 10)

The four prior papers can be read as the structural derivation that the present paper takes as substrate. Each named one property that the journal had to possess for the program to live in it; the present paper takes the four properties as given and reads off the four operational equivalences that follow. Where each prior paper went from one structural commitment to its consequences, the present paper goes from four structural commitments to a single primitive — replay — under which the four canonical operational disciplines collapse. The relation is therefore not parallel: §4.1’s substrate is what the prior four established, and §4.2’s theorem is what follows once it is in hand.

9. Counter-arguments

The substrate condition and its four equivalences invert vocabulary the canonical literature has used productively for decades. A reader familiar with that literature is likely to raise objections that are not addressed by §§2–8. Four are taken up here. The treatment of each follows the same shape: name the valid aspect of the objection, then identify the structural premise on which the

invalid aspect depends and observe that the substrate condition removes that premise. The refutations are made from the construct, not from any framework that instantiates it.

9.1 “Loading a large journal takes time”

Canonical assumption. Bringing a system to a serving state requires reconstructing its persisted history end-to-end, and the cost of that reconstruction grows without bound with the system’s operational age.

The objection. A journal that accumulates entries over the operational life of a program grows without bound. Bringing a new instance to a serving state by replaying the journal therefore takes longer the older the program becomes. A canonical anecdote names a relational system whose initial load — from a binary log accumulated over a year — required roughly an hour before the system could accept traffic. If deployment is replay (§5.1), then deployment is bounded below by that load time, and the substrate’s account of deployment makes the deploy window worse, not better.

What is valid. Replay is linear in the entries it reads. A journal twice as long takes twice as long to replay. The objection correctly observes that the absolute deploy time grows with the program’s history under any substrate-based account of deployment.

What does not follow. The hour-long load named in the anecdote is not replay over the substrate condition; it is reconstruction of state from a log whose entries are state-change deltas. The two régimes have different cost basis. Under the substrate condition, the entry is a compact named operation — `ActionId` plus arguments — and replay is the application of those operations in order (Papers 1 and 2). The per-entry cost is the in-memory apply of one operation, not the disk-bound write or schema-mediated commit that state-change replay incurs.

§5.1’s measurement materialises the structural distinction. L1 reports a sustained replay rate of **451 thousand entries per second** at the median ($N = 100\,000$) and **416 thousand entries per second** at $N = 1\,000\,000$ — the rate is bounded by neither N nor the per-entry apply cost, and the orchestration tail around bulk replay is below $30\,\mu\text{s}$ at every measured N . A journal of ten million compact entries replays in the order of twenty seconds at this rate, not an hour. The objection generalises a measurement made in one régime — state replay over a binary log — to a régime in which the entries are operations, and the generalisation does not hold. What grows linearly is the number of operations the program has uttered; what does not grow is the cost of uttering each one again in memory.

One canonical case lies outside this argument and is preserved as an honest limit. Where the cost-dominant operation at a single entry is itself expensive — a network round-trip to an external service, a complex domain calculation — replay carries that cost per entry just as live operation did. The substrate

does not make any one operation cheaper; it makes the bookkeeping around the operation negligible.

9.2 “Change-data-capture already solves replication”

Canonical assumption. Cross-datacenter replication is properly addressed by reconstructing events from the originating store’s state-change deltas, mediated by a CDC pipeline between primary and replica.

The objection. Cross-datacenter replication is a mature engineering concern. Change-data-capture pipelines, log shipping between database engines, and managed replication services route writes from primary to replica with bounded lag and well-understood failure modes. The substrate’s account of replication (§5.2) appears to re-derive a problem that the canon has solved repeatedly. What additional structural claim does the substrate condition support that the existing apparatus does not?

What is valid. CDC pipelines work. Production systems replicate across datacenters with CDC every day; the apparatus is reliable, vendor-supported, and well-tooled. As an engineering solution, it solves the operational concern of moving state between sites.

What does not follow. The structural premise of CDC is that the originating store records state changes, not operations, and that events must be reconstructed from those state changes by inferential reading of the binary log (§2.2, §6.2). The reconstruction is lossy: two distinct programmatic operations that produce identical state changes are indistinguishable at the CDC layer. The wire across which CDC traffic travels carries a derived artifact — column-level deltas — at a width that scales with the size of the affected rows rather than with the operation that caused them.

The substrate condition replaces the reconstruction with direct emission. The journal entry is the program’s operation, in the language of the program (Paper 1), reduced to a reference to a parametric verb plus its arguments (Paper 2). The wire carries the operation as uttered, not the state changes it produced. §5.2’s measurement (L2) reports a compact wire of **36 to 99.7 bytes per event** depending on verb tier, against literal-form payloads of **121 to 913.7 bytes**; the literal-to-compact ratio reaches **9.2× at the production-shaped tier and 20× at full production scale** (Paper 2 Lab 4). Applying `gzip` to the literal form closes the gap only to **3.9× at tier 3** — the structural saving is not algorithmic but representational, and a compressor operating after the fact cannot recover what naming achieved by construction.

The objection is therefore precise in scope. CDC solves replication as state propagation. The substrate condition does not deny that solution; it shows that under the construct’s conditions the problem CDC solves does not arise, and the wire carries a different artifact. The two are not in competition for the same engineering decision; they are at different layers of the design.

9.3 “Snapshot backup is simpler”

Canonical assumption. Backup is a captured image of state at a chosen moment, archived by an external apparatus and re-instantiated at recovery time.

The objection. Backup as a periodic state snapshot is a forty-year-old discipline with well-understood operational properties: recovery time objective (RTO), recovery point objective (RPO), storage tiering, lifecycle policies. The substrate’s account of backup (§5.3) requires a continuously consuming destination — a process that holds the corpus as it grows — which appears operationally more complex than periodic state capture. Why prefer continuous to periodic?

What is valid. Periodic snapshots have an operational vocabulary the field knows. The discipline is mature, vendor-supported, and integrates with object-store lifecycle infrastructure. As an engineering choice for systems whose program is reducible to state, the snapshot apparatus is a sound default.

What does not follow. The structural premise of snapshot backup is that the program is reducible to its state — that a captured image at moment t suffices to characterise the program at that moment (§6.3). The premise holds for programs whose journal is, in effect, a derivative of state changes. It does not hold for programs whose journal is the program itself.

Under the substrate condition, the artifact that characterises the program is the journal, not any state derived from it. A snapshot at moment t captures *where* the program had arrived; the corpus to t captures *how it got there*. The two are not equivalent informational artifacts: from the snapshot, the operational history is lost; from the corpus, the snapshot at any past moment is recoverable by replay. The substrate-based account of backup is therefore not simply a different operational pattern with the same content; it is the strictly more informative choice when the construct’s conditions hold.

The cost is honest. A continuously consuming destination, even one whose role is to hold the corpus and never replay it, must be reachable, must persist what arrives, and must catch up when it falls behind. §5.3’s measurement (L4) reports steady-state convergence of **2 419 events/sec on FileSystem, 1 004 events/sec on MySQL, and 764 events/sec on SQL Server**, with per-instance verification reaching equal in-memory state in **18 of 18 cells across three backends and three journal sizes**. The numbers establish that the operational obligation is bounded — a destination that ingests at the rates above is not a degenerate case — and that the corpus arrives identically across heterogeneous backends. The objection’s premise that simplicity favours snapshot is honest engineering; the construct’s reply is that the simpler artifact carries less information, and the choice between holding state and holding program is the choice the substrate condition makes available.

9.4 “Append does not scale beyond N writes/sec”

Canonical assumption. A journaled append is bounded by the per-store write throughput of the underlying medium, and that ceiling constrains any substrate-based account to per-actor write rates the medium can absorb.

The objection. Every persistent store has a write-throughput ceiling. A substrate that records every operation as a journal append inherits whatever ceiling the underlying store imposes; production systems that exceed that ceiling are constrained either to batched-write modes or to relaxed-durability commits. The substrate condition therefore does not generalise to high-throughput workloads, and the four equivalences hold only for systems that fit within the per-actor write rate the journal can absorb.

What is valid. A local-journal substrate, like any persistent store, has a per-entry append latency that bounds its write throughput. §5.5’s measurement (L6) reports **347 μ s p50 on the local FileSystem backend** under one-entry one-durable-commit semantics, corresponding to roughly three thousand entries per second of single-writer append rate. A workload that exceeds that rate on a single actor cannot be absorbed by single-writer append at the same durability guarantee.

What does not follow. The single-actor ceiling is not a ceiling on the construct; it is a property of the configuration in which one actor handles one event stream. The substrate condition is preserved under sharding (one actor per shard, each with its own journal — the per-actor invariant of Paper 3, §6.2 and §6.8) and under local buffering with asynchronous replication to a canonical backend (§5.4’s offline equivalence applies to the steady-state régime, not only to outage absorption). Neither sharding nor buffering is foreign to the construct; both fall out of the substrate’s locality property — the journal is the actor’s, not the system’s.

The honest scope is therefore narrower than the objection suggests. The construct does not claim that a single actor with one journal absorbs unbounded write rates; it claims that within the rate the journal can absorb at a given durability guarantee, the four equivalences hold. §5.5’s table records that the local-journal régime favours the substrate by **3 $\times$ over both co-located RDBMS engines at p50 and at the long tail**, and that the advantage is structural (both RDBMS engines land in the same band). The trade-off is not journal-vs-no-journal; it is local-append-vs-network-commit, and the substrate’s ceiling is the higher of the two before any sharding is considered.

Relaxing durability changes the constant. Group commit, batched inserts, and asynchronous-durability modes on the RDBMS side close part of the gap by trading durability for throughput; the substrate’s local-append rate can be raised similarly by group-flush semantics. §5.5’s caveats name both choices as honest engineering. What the comparison preserves under all durability régimes is the structural claim: replay-based operations inherit append latency as their dominant cost term, and a local append against a substrate is, in every measured

régime, at least as fast as a comparable commit against a co-located transactional store.

10. Conclusion

The canonical literature on distributed systems has long treated deployment, cross-datacenter replication, backup, and offline operation as four operational disciplines, each supported by its own apparatus. This treatment proved productive: blue-green environments, change-data-capture pipelines, snapshot agents, and message-queue buffers matured into reliable engineering practice. From that productivity grew the fragmentation analysed in §2 and §6 — four parallel literatures, four parallel apparatuses, and four parallel cost models, borne independently by systems that must be zero-downtime, cross-datacenter, recoverable, and tolerant of disconnection.

This paper observes that the fragmentation is not entailed by the operations themselves. It follows from the assumption named in §3: that operational concerns are addressed by apparatus around the program rather than by the program itself. Under the substrate condition established by prior papers in this series, that assumption no longer holds. The journal is the program written out over time, and four equivalences follow from reading it under variations of when and where (§4.2): deployment is replay at the present head; replication is the same replay at another site; backup is the corpus held without replay; offline operation is replay after a delay.

Viewed from this condition, the apparatuses surveyed in §6 appear as mechanisms that reconstruct, from outside the program, work that the substrate already performs by construction. Blue-green deployment reconstructs replay manually; change-data-capture reconstructs operations from state; snapshot backup captures a derivative of the program’s history; message-queue buffers introduce a durable artefact to absorb disconnection that the substrate’s own persistence already absorbs.

The contribution of this paper is conceptual. The instantiation in production serves as an existence proof; the measurements in §5 exhibit the régime in which the four equivalences are observable. One artefact — the substrate — supports four operational readings without requiring separate mechanisms.

For a journaled program, deployment is replay, replication is sharing history, backup is copying the program, and offline operation is delayed replay. The four operational disciplines named in §2 remain meaningful; what changes is the structural locus of operational concerns: from externally-added apparatus to internal readings of the program’s substrate.

Appendix A. Source-code verification

The constructs named in §7 are publicly available in the Puppeteer codebase. The line references below are pinned against the master branch as of the drafting of this section. Three tables organise the surface by sub-section: the *aliveGate* mechanism (§7.1), the push-to-pull primitives (§7.2), and the *Materialize* construct (§7.3).

§7.1 — Theater.aliveGate

Surface	File	Line(s)	What it shows
<code>aliveGate</code> manual-reset event	<code>Choreography/Theater/Performance.c</code>	<code>341</code>	Reset while follower or during handover; Set when alive
<code>Start(bool asFollower = false)</code>	<code>Choreography/Theater/Performance.c</code>	<code>100</code>	Two-mode start; sets gate iff primary
<code>WaitUntilAlive(CacheTagEntry)</code>	<code>Choreography/Theater/Performance.c</code>	<code>136</code>	External callers block here
<code>IsAlive</code> predicate	<code>Choreography/Theater/Performance.c</code>	<code>167</code>	False while follower; true after handover
<code>LockWhileNotSynchronized()</code>	<code>Choreography/Theater/Performance.c</code>	<code>177</code>	Pause writes on primary; return <code>EntryId</code>
<code>UnlockAndRunAlive()</code>	<code>Choreography/Theater/Performance.c</code>	<code>183</code>	Flip gate; release primary
<code>CatchUpFromJournal(Puppeteer/EventSourcing/ActorHandler/Replay(targetEntryId))</code>	<code>Puppeteer/EventSourcing/ActorHandler/Replay.c</code>	<code>179</code>	Replay residual tail under write lock
<code>RehydrateFromEvent(Puppeteer/EventSourcing/DB/Diary.c)</code>	<code>Puppeteer/EventSourcing/DB/Diary.c</code>	<code>228</code>	Diary-side bulk replay primitive

§7.2 — Push-to-pull primitives

Surface	File	Line(s)	What it shows
<code>Diary.OnRecordWrite</code> setter	<code>Puppeteer/EventSourcing/DB/Diary.c</code>	<code>145</code>	Per-record callback (single subscriber)

Surface	File	Line(s)	What it shows
AddRecordWrittenCallback	Puppeteer/EventSourcing/DB/Diary.cs	160	Chain N subscribers without displacing existing
ReactionMode.Cue	Puppeteer/EventSourcing/Follower/Reactions.cs	19	Reaction push-mode
Cue batch → push loop wiring	Puppeteer/EventSourcing/Follower/Reactions.cs	563-579	Reaction catch then subscribe to callback
SVIX integration	—	—	Not present in Puppeteer/ or Choreography/; wired at the per-API host process

§7.3 — Materialize construct

Surface	File	Line(s)	What it shows
Performance.StartChoreography(TheaterPerformance>true)	Choreography/TheaterPerformance.cs	100	Alternate startup mode; SuppressReactionJournaling flag
Reaction.MaterializeFromMirror	Puppeteer/EventSourcing/Follower/Reactions.cs	660	DSL markers plumbing: records destination on action
actor.Materialize sub-namespaces	Puppeteer/Materialize/actor.cs	14	Register / Deregister / List / ReadRecordsAfter / ConfirmUntil / ReadReactions / ReadElidedRange
MaterializeMirror fluent client	Puppeteer/Materialize/mirror.cs	14	Sync() (Capa 1); AsProgramMirror().Sync() (Capa 2)

Surface	File	Line(s)	What it shows
MirrorSyncResult immutable struct	Puppeteer/MaterializeError.cs	10-202	Records / ReactionsSnapshot / ElisionMarkers + watermark advance
Diary backend polymorphism	Puppeteer/EventSource/DB/Diary.cs	53-78	DatabaseType, connectionString) → InMemory / FileSystem / MySQL / SQLServer
Local-buffer (offline régime)	Puppeteer/EventSource/DB/Diary.cs	28-34, 80-85	IsBuffered; ReplicationAgent async drain to canonical storage

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

Appendix B. Bibliography

Online sources accessed at the moment of drafting; dates updated at release.

- Ambler, S. W., & Sadalage, P. J. (2006). *Refactoring Databases: Evolutionary Database Design*. Addison-Wesley. ISBN 978-0-321-29353-5.
- AWS documentation. *Managing your storage lifecycle*. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-mgmt.html> (accessed 2026-05-14).
- Azure documentation. *Overview of operational backup for Azure Blobs*. <https://learn.microsoft.com/en-us/azure/backup/blob-backup-overview> (accessed 2026-05-14).
- Bacula project. <https://www.bacula.org/> (accessed 2026-05-14).
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. Communications of the ACM, 59(5), 50–57.
- Debezium project. <https://debezium.io/> (accessed 2026-05-14).
- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007).

- Dynamo: Amazon's Highly Available Key-value Store*. Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07), 205–220.
- Flyway project. <https://flywaydb.org/> (accessed 2026-05-14).
 - Fowler, M. (2010). *BlueGreenDeployment*. [martinfowler.com/bliki. https://martinfowler.com/bliki/BlueGreenDeployment.html](https://martinfowler.com/bliki/BlueGreenDeployment.html) (accessed 2026-05-14).
 - Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). *Design science in information systems research*. MIS Quarterly, 28(1), 75–105.
 - Hitz, D., Lau, J., & Malcolm, M. (1994). *File System Design for an NFS File Server Appliance*. Proceedings of the USENIX Winter Technical Conference, San Francisco, January 1994.
 - Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley. ISBN 978-0-321-60191-9.
 - Kreps, J., Narkhede, N., & Rao, J. (2011). *Kafka: a Distributed Messaging System for Log Processing*. Proceedings of the NetDB Workshop, 6th International Workshop on Networking Meets Databases (co-located with SIGMOD 2011).
 - Kubernetes documentation. *Deployments — Rolling Update Deployment Strategy*. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/#rolling-update-deployment> (accessed 2026-05-14).
 - Lakshman, A., & Malik, P. (2010). *Cassandra: A Decentralized Structured Storage System*. ACM SIGOPS Operating Systems Review, 44(2), 35–40.
 - Liquibase project. <https://www.liquibase.org/> (accessed 2026-05-14).
 - Maxwell project. *Maxwell's Daemon*. <https://maxwells-daemon.io/> (accessed 2026-05-14).
 - MySQL Reference Manual. *Chapter 17: Replication*. <https://dev.mysql.com/doc/refman/8.0/en/replication.html> (accessed 2026-05-14).
 - PostgreSQL documentation. *Continuous Archiving and Point-in-Time Recovery (PITR)*. <https://www.postgresql.org/docs/current/continuous-archiving.html> (accessed 2026-05-14).
 - RabbitMQ documentation. *Dead Letter Exchanges*. <https://www.rabbitmq.com/dlx.html> (accessed 2026-05-14).
 - Redis documentation. *Redis replication*. <https://redis.io/docs/management/replication/> (accessed 2026-05-14).
 - Sato, D. (2014). *CanaryRelease*. [martinfowler.com/bliki. https://martinfowler.com/bliki/CanaryRelease.html](https://martinfowler.com/bliki/CanaryRelease.html) (accessed 2026-05-14).
 - Veeam documentation. *Veeam Backup & Replication User Guide*. <https://helpcenter.veeam.com/docs/backup/vsphere/overview.html> (accessed 2026-05-14).
-
- Rivera, A. (2026). *Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems*. Paper 1 of this series. 01-anti-porosity.md

- Rivera, A. (2026). *Program-value separability: the structural precondition for compilation, caching, and dense journaling in a DSL runtime*. Paper 2 of this series. 02-program-value-separability.md
- Rivera, A. (2026). *Reactions and the partition: opt-in eventual consistency in actor-native systems*. Paper 3 of this series. 03-reactions-and-partition.md
- Rivera, A. (2026). *Preserving semantic continuity across actors: a tell-based approach without orchestration*. Paper 4 of this series. 04-cross-actor-continuity.md